

</>

The Anuj DECODED

</>

```

01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};

```



```

01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver

```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3903 · Smallest Stable Index I

LeetCode · Daily Question

Easy

Array

Prefix / Suffix

1. What Type of Question Is This?

This is a classic **prefix / suffix aggregation problem**. At every index i , the problem asks you to compare two running aggregates computed over disjoint parts of the array: the **maximum of everything up to and including i** , and the **minimum of everything from i to the end**. Whenever a problem defines a per-index score using "max/min of the prefix" and "max/min of the suffix" together, the standard toolkit is to **precompute a prefix array and a suffix array once**, rather than recomputing either aggregate from scratch at every index.

2. What Is Given?

- An integer array **nums** of length n .
- An integer **k**.
- For each index i , its **instability score** is defined as: $\max(\text{nums}[0..i]) - \min(\text{nums}[i..n-1])$.

$\max(\text{nums}[0..i])$	The largest value among the elements from index 0 to index i (inclusive) — the running maximum
$\min(\text{nums}[i..n-1])$	The smallest value among the elements from index i to the end of the array — the running minimum
stable index	An index i whose instability score is $\leq k$.

3. What Do We Need to Find?

Return the **smallest** index i whose instability score is $\leq k$. If no index qualifies, return **-1**.

Function Signature

■ C++

```
class Solution {
public:
    int firstStableIndex(vector<int>& nums, int k) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  nums = [5,0,1,4], k = 3
Output: 3
```

Index 0: $\max([5]) = 5$, $\min([5,0,1,4]) = 0$, score = $5 - 0 = 5$ (too big). Index 1: $\max([5,0]) = 5$, $\min([0,1,4]) = 0$, score = 5 (too big). Index 2: $\max([5,0,1]) = 5$, $\min([1,4]) = 1$, score = 4 (still too big). Index 3: $\max([5,0,1,4]) = 5$, $\min([4]) = 4$, score = 1 , which is $\leq k = 3$. This is the first index that qualifies $\rightarrow$ **3**.

Example 2 — no valid index

```
Input:  nums = [3,2,1], k = 1
Output: -1
```

Every index here gives a score of $3 - 1 = 2$ (the overall max is always 3, the overall min is always 1, no matter where you split). Since $2 > k = 1$ for every index, none qualifies $\rightarrow$ **-1**.

Example 3 — single element

```
Input:  nums = [0], k = 0
Output: 0
```

With only one element, $\max([0]) = \min([0]) = 0$, so the score at index 0 is $0 - 0 = 0$, which is $\leq k = 0 \rightarrow$ **0**.

Constraints

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$
- $0 \leq k \leq 10^9$

n is tiny here (≤ 100), so even an $O(n^2)$ approach comfortably passes — but the $O(n)$ prefix/suffix technique below is the one worth remembering, since it's the standard tool for this whole family of problems regardless of how large n gets.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Notice the two aggregates move in opposite directions.** $\max(\text{nums}[0..i])$ only ever grows (or stays the same) as i increases, because you're adding more elements to the window. $\min(\text{nums}[i..n-1])$ only ever grows (or stays the same) too, because you're removing elements from the left, which can only raise the minimum or leave it unchanged. Both are individually monotonic — but their *difference* (the instability score) isn't necessarily monotonic, so you can't binary search on it directly.
- **Step 2 — Recognize the recomputation waste.** A naive solution recalculates $\max(\text{nums}[0..i])$ and $\min(\text{nums}[i..n-1])$ from scratch for every i , scanning up to n elements each time — that's $O(n)$ work repeated n times.
- **Step 3 — Precompute instead of recompute.** Build a **prefix-max array** left to right ($\text{prefixMax}[i] = \max(\text{prefixMax}[i-1], \text{nums}[i])$) and a **suffix-min array** right to left ($\text{suffixMin}[i] = \min(\text{suffixMin}[i+1], \text{nums}[i])$), each in a single $O(n)$ pass.
- **Step 4 — Scan once for the answer.** With both arrays ready, $\text{score}(i) = \text{prefixMax}[i] - \text{suffixMin}[i]$ is an $O(1)$ lookup per index. Walk i from 0 upward and return the first index where the score is $\leq k$.
- **Step 5 — Trim to the essentials.** The suffix-min array is unavoidable (you need to know minimums for indices *ahead of* where you currently are before you get there), but the prefix-max array can be folded into the same final scan as a single running variable instead of its own array — you're only ever comparing against the running max so far, which you can track as you go.

Which Algorithm Is Best?

The **optimal** approach precomputes only the suffix-min array (unavoidable, since it depends on elements you haven't reached yet in a left-to-right scan), then does one final left-to-right pass tracking the prefix maximum as a running variable and checking the stability condition immediately, returning as soon as the first stable index is found. That's $O(n)$ time with roughly half the extra memory of the two-array version, and it's the natural end point of this progression — the brute-force and "better" tiers below show the steps to get there.

Approach 1 — Brute Force (Recompute Max and Min at Every Index)

Idea: Translate the definition literally: for every index i , scan from 0 to i to find the prefix maximum, then scan from i to the end to find the suffix minimum, and check the stability condition. No memory of previous computations is kept between indices.

Steps

- Loop i from 0 to $n-1$.
- For the current i , scan indices 0.. i to find $\max(\text{nums}[0..i])$.
- Scan indices $i..n-1$ to find $\min(\text{nums}[i..n-1])$.
- If the difference is $\leq k$, return i immediately — it's the smallest such index by construction, since we're scanning left to right.
- If no index qualifies after the full loop, return -1.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int firstStableIndex(vector<int>& nums, int k) {
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            int mx = INT_MIN;
            for (int a = 0; a <= i; a++) mx = max(mx, nums[a]); // max(nums[0..i])

            int mn = INT_MAX;
            for (int b = i; b < n; b++) mn = min(mn, nums[b]); // min(nums[i..n-1])

            if ((long long)mx - mn <= k) return i;
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {5, 0, 1, 4};
    int k = 3;
    Solution sol;
    cout << sol.firstStableIndex(nums, k) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def firstStableIndex(self, nums, k):
        n = len(nums)
        for i in range(n):
            mx = max(nums[0:i + 1]) # max(nums[0..i])
            mn = min(nums[i:n])     # min(nums[i..n-1])
            if mx - mn <= k:
                return i
        return -1

if __name__ == "__main__":
    nums = [5, 0, 1, 4]
```

```
k = 3
print(Solution().firstStableIndex(nums, k))    # Expected output: 3
```

■ JavaScript

```
class Solution {
  firstStableIndex(nums, k) {
    const n = nums.length;
    for (let i = 0; i < n; i++) {
      let mx = -Infinity;
      for (let a = 0; a <= i; a++) mx = Math.max(mx, nums[a]);    // max(nums[0..i])

      let mn = Infinity;
      for (let b = i; b < n; b++) mn = Math.min(mn, nums[b]);    // min(nums[i..n-1])

      if (mx - mn <= k) return i;
    }
    return -1;
  }
}

const nums = [5, 0, 1, 4];
const k = 3;
console.log(new Solution().firstStableIndex(nums, k));    // Expected output: 3
```

Complexity: For each of the n indices, both inner scans together touch up to n elements, giving $O(n)$ work per index and $O(n^2)$ overall. With $n \leq 100$ this is only $\sim 10,000$ operations and runs instantly here, but it re-derives the same running max/min from scratch at every step instead of reusing previous work.

Approach 2 — Better (Precompute Both Prefix-Max and Suffix-Min Arrays)

Idea: Stop recomputing the same running aggregates over and over. Build a prefix-max array in one left-to-right pass and a suffix-min array in one right-to-left pass, then every index's instability score becomes a plain $O(1)$ array lookup and subtraction.

Steps

- Build prefixMax[] left to right: prefixMax[0] = nums[0], and prefixMax[i] = max(prefixMax[i-1], nums[i]) for $i > 0$.
- Build suffixMin[] right to left: suffixMin[n-1] = nums[n-1], and suffixMin[i] = min(suffixMin[i+1], nums[i]) for $i < n-1$.
- Scan i from 0 to n-1 and check prefixMax[i] – suffixMin[i] ≤ k.
- Return the first i where this holds; return -1 if the loop finishes without one.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int firstStableIndex(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> prefixMax(n), suffixMin(n);

        prefixMax[0] = nums[0];
        for (int i = 1; i < n; i++)
            prefixMax[i] = max(prefixMax[i - 1], nums[i]);

        suffixMin[n - 1] = nums[n - 1];
        for (int i = n - 2; i >= 0; i--)
            suffixMin[i] = min(suffixMin[i + 1], nums[i]);

        for (int i = 0; i < n; i++) {
            if ((long long)prefixMax[i] - suffixMin[i] <= k) return i;
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {5, 0, 1, 4};
    int k = 3;
    Solution sol;
    cout << sol.firstStableIndex(nums, k) << endl;    // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def firstStableIndex(self, nums, k):
        n = len(nums)

        prefixMax = [0] * n
        prefixMax[0] = nums[0]
        for i in range(1, n):
            prefixMax[i] = max(prefixMax[i - 1], nums[i])

        suffixMin = [0] * n
```

```

    suffixMin[n - 1] = nums[n - 1]
    for i in range(n - 2, -1, -1):
        suffixMin[i] = min(suffixMin[i + 1], nums[i])

    for i in range(n):
        if prefixMax[i] - suffixMin[i] <= k:
            return i
    return -1

if __name__ == "__main__":
    nums = [5, 0, 1, 4]
    k = 3
    print(Solution().firstStableIndex(nums, k))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    firstStableIndex(nums, k) {
        const n = nums.length;

        const prefixMax = new Array(n);
        prefixMax[0] = nums[0];
        for (let i = 1; i < n; i++) prefixMax[i] = Math.max(prefixMax[i - 1], nums[i]);

        const suffixMin = new Array(n);
        suffixMin[n - 1] = nums[n - 1];
        for (let i = n - 2; i >= 0; i--) suffixMin[i] = Math.min(suffixMin[i + 1], nums[i]);

        for (let i = 0; i < n; i++) {
            if (prefixMax[i] - suffixMin[i] <= k) return i;
        }
        return -1;
    }
}

const nums = [5, 0, 1, 4];
const k = 3;
console.log(new Solution().firstStableIndex(nums, k));    // Expected output: 3

```

Complexity: Three linear passes (prefix build, suffix build, final scan) each $O(n)$, so $O(n)$ time overall — a big improvement over Approach 1's $O(n^2)$. Uses $O(n)$ extra space for the two auxiliary arrays.

Approach 3 — Optimal (Suffix-Min Array + Prefix Max as a Running Variable)

Idea: The suffix-min array can't be avoided in a single left-to-right pass (you need to know minimums for indices ahead of you before you get there), but the prefix-max array can be dropped entirely — you only ever need the maximum seen *so far*, which is naturally a single running variable updated as you walk left to right, in the very same loop that checks stability and returns the answer.

Steps

- Build suffixMin[] right to left exactly as before — this is the one piece of information that genuinely requires looking ahead.
- Walk i from 0 to n-1 with a single running variable runningMax, updating runningMax = max(runningMax, nums[i]) at each step.
- Immediately check runningMax – suffixMin[i] ≤ k and return i the moment it holds — no separate scan needed.
- Return -1 if the loop completes without finding a stable index.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int firstStableIndex(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> suffixMin(n);
        suffixMin[n - 1] = nums[n - 1];
        for (int i = n - 2; i >= 0; i--)
            suffixMin[i] = min(suffixMin[i + 1], nums[i]);

        int runningMax = INT_MIN;
        for (int i = 0; i < n; i++) {
            runningMax = max(runningMax, nums[i]); // prefix max, tracked on the fly
            if ((long long)runningMax - suffixMin[i] <= k) return i;
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {5, 0, 1, 4};
    int k = 3;
    Solution sol;
    cout << sol.firstStableIndex(nums, k) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def firstStableIndex(self, nums, k):
        n = len(nums)

        suffixMin = [0] * n
        suffixMin[n - 1] = nums[n - 1]
        for i in range(n - 2, -1, -1):
            suffixMin[i] = min(suffixMin[i + 1], nums[i])
```

```

        runningMax = float("-inf")
        for i in range(n):
            runningMax = max(runningMax, nums[i])    # prefix max, tracked on the fly
            if runningMax - suffixMin[i] <= k:
                return i
        return -1

if __name__ == "__main__":
    nums = [5, 0, 1, 4]
    k = 3
    print(Solution().firstStableIndex(nums, k))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    firstStableIndex(nums, k) {
        const n = nums.length;

        const suffixMin = new Array(n);
        suffixMin[n - 1] = nums[n - 1];
        for (let i = n - 2; i >= 0; i--) suffixMin[i] = Math.min(suffixMin[i + 1], nums[i]);

        let runningMax = -Infinity;
        for (let i = 0; i < n; i++) {
            runningMax = Math.max(runningMax, nums[i]);    // prefix max, tracked on the fly
            if (runningMax - suffixMin[i] <= k) return i;
        }
        return -1;
    }
}

const nums = [5, 0, 1, 4];
const k = 3;
console.log(new Solution().firstStableIndex(nums, k));    // Expected output: 3

```

Complexity: Still $O(n)$ time (one backward pass to build `suffixMin`, one forward pass to answer), but only **one** auxiliary array instead of two, and the final scan does its own max-tracking and stability check together instead of in two separate loops — fewer operations and less memory than Approach 2 for the same asymptotic class.

Side-by-Side Comparison

1. Brute Force	Recompute prefix max and suffix min from scratch at every index	$O(n^2)$	$O(1)$
2. Better	Precompute full prefix-max and suffix-min arrays, then scan once	$O(n)$	$O(n)$
3. Optimal	Precompute suffix-min only; track prefix max as a running variable	$O(n)$	$O(n)$, roughly half of Approach 2

Key Takeaways to Remember

- **"Max/min of a prefix" and "max/min of a suffix" together** is one of the most common array patterns — default to precomputing a prefix array and a suffix array in $O(n)$ instead of recomputing either from scratch per index.
- A running aggregate you only ever need *up to the current point* (like the prefix max here) doesn't need its own array at all — a single variable updated inside your main loop is enough and saves memory.
- An aggregate you need *ahead of* where your main loop currently is (like the suffix min here) genuinely needs to be precomputed first, since you can't know it without looking forward.
- Individually monotonic prefix/suffix aggregates don't guarantee their *difference* is monotonic — don't assume binary search applies just because the two pieces are each well-behaved.
- All three tiers were validated against **Example 1** (nums = [5,0,1,4], k = 3 → 3) across C++, Python, and JavaScript, and checked against Examples 2 and 3 by hand in the walkthrough above.